LLM-Assisted Automatic Memetic Algorithm for Lot-Streaming Hybrid Job Shop Scheduling With Variable Sublots

Rui Li, Ling Wang, *Senior Member, IEEE*, Hongyan Sang *Member, IEEE*, Lizhong Yao, and Lijun Pan

***Abstract*— This study addresses the lot-streaming hybrid job shop scheduling problem with variable sublots (LHJSV), inspired by a real-world aircraft tooling shop. A computational model is developed to represent the complex scheduling processes of the tooling shop. To solve this problem, we propose an automatic memetic algorithm enhanced by a heuristic designed with the assistance of a large language model (LLM). The approach is designed as follows: first, a memetic computing framework with automated algorithmic design is proposed for LHJSV. Second, a cooperative evolutionary heuristic framework based on problem decomposition is introduced, enabling the LLM to comprehend the LHJSV characteristics and generate feasible algorithms. Third, problem-specific prompts for LHJSV are carefully designed to guide the LLM. To evaluate the effectiveness of the proposed method, 20 benchmark instances derived from the Taillard dataset and a real-world case involving 575 operations are utilized. The proposed algorithm is compared against three swarm-based algorithms, an end-to-end method, and an LLM-based algorithm. Experimental results demonstrate that our method outperforms the compared algorithms on 85% of benchmark instances and exhibits significant superiority in real-world scenarios.**

***Index Terms*— Automatic algorithm design, evolutionary computation, large language model (LLM), lot-streaming scheduling, memetic computing.**

I. Introduction

AIRCRAFT manufacturing is a critical national industry, reflecting a nation's ability to independently design and produce high-quality air carriers. With the continued growth of global manufacturing, the number of large aircraft in service worldwide is projected to reach 48 230 in 2043 [1]. Consequently, rapidly increasing production capacity has become a major priority for aircraft manufacturers. Enhancing efficiency through advanced scheduling systems, particularly from a resource allocation perspective, remains the most effective approach. However, the aircraft scheduling is highly complex, involving intricate working conditions, extensive manual work, human-machine collaboration, uncertainty, and dynamics constraints [2], making it challenging to model and optimize. Tooling shops play a crucial role in manufacturin essential aircraft components, such as brackets, metal sheets, baffles, and other structural parts. These operations are characterized by batch processing, variable operation sequences, flexible sublot sizes, and parallel machine environments, adding to the scheduling complexity. As a result, the problem can be formulated as the lot-streaming hybrid job shop scheduling with variable sublots (LHJSV) [3].

The LHJSV extends the classical job scheduling problem, which is well-known to be NP-hard [4]. It encompasses three interrelated subproblems: 1) operation sequencing (OS); 2) lot-splitting (LS); and 3) sublot machine selection (MS). Solving the LHJSV poses significant challenges, as its solution space grows exponentially with increasing problem scale, making it computationally intractable to solve within a short time frame. Furthermore, the LHJSV studied in this work introduces additional complexities compared to traditional lot-streaming scheduling with variable sublots (LSV) [5]. In conventional LSV, the sublot size for each operation may vary, but each type of machine is limited to one unit. Once a sublot is completed, it can proceed to the next operation without waiting for the completion of other sublots from the same lot. In contrast, the LHJSV allows sublot sizes to dynamically change at each operation while also accommodating multiple parallel machines for each machine type. Moreover, unlike LSV, the next operation in LHJSV can only begin once all sublots of the preceding operation are completed. Compared to multiprocessor shop scheduling (MPSS), where lots are equally divided among processors [6], LHJSV introduces additional complexities, including integer-based sublot division and adaptive MS, which impose stricter constraints and make it inherently more challenging than both LSV and MPSS. To minimize the makespan, it is essential to develop efficient scheduling strategies that integrate OS, LS, and parallel MS under intricate constraints. Addressing these challenges holds

Received 14 January 2025; revised 7 March 2025 and 25 March 2025; accepted 26 March 2025. Date of publication 31 March 2025; date of current version 31 July 2026. This work was supported in part by the National Key Research and Development Project under Grant 2023YFB3308002; in part by the National Natural Science Foundation of China under Grant 62273193 and Grant 62473186; and in part by the National Natural Science Foundation of China Regional United Development Fund under Grant U24A20273. *(Corresponding authors: Ling Wang; Hongyan Sang.)*

Rui Li and Ling Wang are with the Department of Automation, Tsinghua University, Beijing 100084, China (e-mail: li-r23@mails.tsinghua.edu.cn; wangling@mail.tsinghua.edu.cn).

Hongyan Sang is with the School of Computer Science, Liaocheng University, Liaocheng 252000, China (e-mail: sanghongyan@lcu-cs.com).

Lizhong Yao is with the College of Physics and Electronic Engineering, Chongqing Normal University, Chongqing 401331, China (e-mail: lizhong_yao@cqnu.edu.cn).

Lijun Pan is with the School of Management, Hunan Institute of Engineering, Xiangtan 411104, China (e-mail: pansoftware@126.com).

This article has supplementary downloadable material available at https://doi.org/10.1109/TEVC.2025.3556186, provided by the authors.

Digital Object Identifier 10.1109/TEVC.2025.3556186

1941-0026 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

significant promise for advancing the productivity and operational efficiency of aircraft manufacturing processes.

Classical strategies for lot-streaming encompass equal sublots [7], consistent sublots [8], and variable sublots [9]. Among these, methods with fixed sublot sizes constrain production efficiency improvements. Consequently, LSV sublot sizes, offering a more flexible and efficient production mode, has garnered significant attention. The primary challenge in variable lot-streaming (LSV) lies in optimally dividing lots and determining efficient operation sequences. Existing methods for tackling LSV can be categorized into the following six strategies: 1) *Exact Methods:* Approaches, such as linear programming [10] and constraint programming [11] yield optimal solutions for small-scale instances but suffer from poor scalability. 2) *Heuristic Methods:* Techniques like iterative greedy search [9] and simulated annealing [12] offer computational efficiency but often suffer from instability and premature convergence. 3) *Neighborhood Search:* Expert-designed methods [3] leverage domain knowledge but require significant manual effort. 4) *Meta-heuristic Methods:* Algorithms, such as genetic algorithms (GAs) [13], particle swarm optimization [14], and artificial bee colony algorithms [15] exhibit limited exploitation capabilities due to the lack of problem-specific search strategies. 5) *Reinforcement Learning (RL):* Value-based approaches like deep Q-networks [7] and policy-based methods leveraging graph neural networks [16] require extensive training, large datasets, and often demonstrate limited generalization. 6) *Hybrid Algorithms:* memetic algorithms (MAs) [17] and meta-heuristics enhanced with RL-based operator selection [8] integrate multiple techniques to enhance performance but involve intricate design processes.

Among these, memetic computing emerges as the most efficient framework, integrating the exploration capabilities of evolutionary algorithms (EAs) with the exploitation strengths of local search and heuristics. However, addressing the LHJSV introduces additional complexities, as it combines the characteristics of both LSV and MPSS. The traditional, artificially designed MAs for LSV, MPSS, and LHJSV struggle to effectively manage these new challenges, because LHJSV entails tightly coupled job shop scheduling, lot size optimization, and machine allocation. Existing problem-specific heuristics require substantial modifications to accommodate these complexities, making their adaptation both knowledge-intensive and time-consuming. Moreover, no single heuristic can consistently perform well across all instances. Consequently, prior research often involves designing multiple heuristics to cooperatively optimize various subproblems, which significantly increases the development cycle. Fortunately, large language models (LLMs) can automatically design and optimize algorithms through iterative search. Guided by search prompts, LLMs can generate superior algorithms compared to human-designed heuristics within a significantly shorter timeframe [30].

The use of LLM for automatic algorithm design has recently become a highly active area of research [18]. LLMs have been applied in optimization tasks through various roles, which can be categorized as follows: 1) *LLM as an Optimizer:* In this role, the LLM directly generates and refines solutions. For instance, Yang [19] fed decision variables and fitness values of solutions to an LLM, which iteratively generated improved solutions. This approach has demonstrated notable improvements in both continuous and combinatorial optimization problems. Additionally, LLMs have been utilized to optimize the mean value of population rankings in evolutionary strategies [20], tune algorithmic parameters [21], and generate new prompts based on initial ones [22]. 2) *LLM as a Predictor:* LLMs can serve as surrogate models to predict the performance of solutions or outcomes in classification or regression tasks. Hao [23] utilized LLMs in this capacity, while Jawahar [24] applied LLMs to estimate the performance of deep neural network architectures. Similarly, Bhambri [25] leveraged LLMs to predict actions in value-based RL tasks. 3) *LLM as an Extractor:* LLMs can extract embedded features from target problems. Wu [26] leveraged LLMs to extract algorithmic features for training an algorithm recommendation system. Jiang [27] employed LLMs to generate embeddings tailored for neural combinatorial optimization tasks. 4) *LLM as a Designer:* In this capacity, LLMs are employed to generate heuristic rules [28], write reward function code for RL [28], formulate mathematical functions [30], and design mutation and selection strategies for GAs [31]. While LLMs have been explored for algorithm design in shop scheduling [28], these studies have primarily focused on simple flow shop scheduling. Their methodologies lack direct applicability to more complex production environments, such as the LHJSV. Directly applying frameworks like those in [28] to LHJSV would result in excessively long and complex prompts. Such complexity may hinder the LLM's ability to accurately interpret the problem structure and generate effective heuristics, thereby reducing its efficacy in addressing the intricate constraints of LHJSV.

In this work, we propose an LLM-assisted MA (LLMMA) that autonomously designs heuristics through cooperative algorithmic evolution for solving the LHJSV. The optimization objective is to minimize the maximum completion time (makespan). The main contributions of this work are summarized as follows.

1) *Problem Level:* The LHJSV problem is abstracted from real-world operations in an aircraft tooling shop, incorporating features, such as lot-streaming, job shop scheduling, parallel machines, and variable sublots. In contrast to traditional LSV and MPSS, LHJSV models real-world complexities, aligning with the specific requirements of aircraft manufacturing.
2) *Methodology Level:* First, we propose a memetic computation framework that leverages LLMs to automatically design heuristics tailored for the LHJSV problem. Second, we develop a cooperative evolutionary framework to decompose and optimize subproblems, effectively handling the constraints of complex shop scheduling. Third, two tailored prompts are designed to address the distinct subproblems of LHJSV, facilitating effective heuristic generation. Fourth, a "code-to-prompt" mechanism is introduced, integrating Chain-of-Thought reasoning to enhance the LLM's interpretability and heuristic generation capabilities.
3) *Domain Level:* In the domain of LSV, our LLM-based MA outperforms state-of-the-art algorithms in terms of solution quality and efficiency. For LLM applications in

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

algorithm design, we introduce a co-evolutionary framework specifically tailored for complex shop scheduling. In shop scheduling, our approach enables the automatic generation of problem-specific heuristics within a MA framework, significantly reducing heuristic development time while improving algorithmic efficiency.

The remainder of this manuscript is organized as follows. Section II introduces the LHJSV problem and its computational model. Section III elaborates on the proposed algorithms, including their design and underlying motivations. Section IV presents the results of numerical experiments and performance analysis. Section V concludes this article with a discussion of findings and future research directions.

II. Problem Description and Modeling

A. Problem Description of LHJSV

An instance of the LHJSV is defined as follows. The problem involves N jobs and N_T types of machines. Each job consists of N_i operations, where $i = 1, \ldots, N$, and each type of machine includes $M_t, t = 1, \ldots, N_T$, parallel machines. For each job, the lot size is denoted as B_i, and all its operations share the same lot size. Each operation $O_{i,j}$, where $i = 1, \ldots, N$ and $j = 1, \ldots, N_i$, selects a type of machine t. The processing time of operation $O_{i,j}$ is represented by $T^P_{i,j}$, while the setup time for each operation, incurred when switching between different operations or sublots, is given as $T^S_{i,j}$. Since additional sublots introduce more setup requirements, the setup time increases proportionally with the number of sublots. The actual processing time of each sublot is expressed as: $T^P_{i,j,s} = H_{i,j,s}T^P_{i,j}/B_i, s = 1, \ldots, R_{i,j}$, where $H_{i,j,s}$ is the proportion of the sublot relative to the total lot size. An operation $O_{i,j}$ can only commence after all sublots of its preceding operation $O_{i,j-1}$ have been completed. The LHJSV problem consists of three primary subproblems: 1) *OS:* Determining the processing order of operations within jobs. 2) *LS:* Dividing each lot into sublots to optimize processing. 3) *Parallel MS:* Allocating each sublot to a suitable parallel machine for processing. The optimization objective is to minimize the makespan, defined as the maximum completion time across all jobs. The following assumptions are introduced: 1) The processing of each sublot cannot be interrupted. 2) The processing resources work the whole day and the processing time of all operations is determined. 3) Dynamic events are not considered.

Table I shows an example of LHJSV with two jobs and two types of machines. The corresponding Gantt chart for Table I is presented in Fig. 1. For job J_1, it comprises two operations, with a lot size of three. Specifically, operation $O_{1,1}$ selects machine type one and is divided into two sublots, with sizes two and one. The processing time of sublot $O_{1,1,1}$ is $(2/3) \times 3 = 2$, and the setup time is 1. Once all sublots of operation $O_{1,1}$ are finished, operation $O_{1,2}$ can start processing.

B. Computational Model

The computational model of LHJSV is built to represent the scheduling sequence and resource allocation of a solution. LHJSV contains the following notations.

TABLE I
Example of LHSJV With Two Jobs and Two Types of Machines

J_i/B_i	$O_{i,j}$	$t/T^P_{i,j}$	N^M_t	$T^S_{i,j}$
$J_1/3$	O_{11}	1/3	2	1
	O_{12}	2/3	3	
$J_2/2$	O_{21}	2/3	3	
	O_{22}	1/2	2	

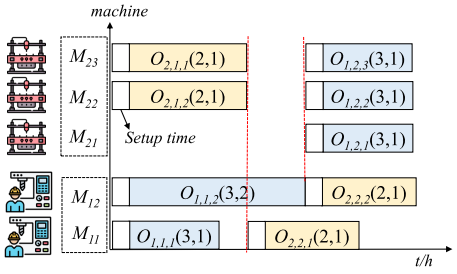


Fig. 1. Gantt chart of the example in Table I.

Indices

i	Job index, $i \in \{1, \ldots, N\}$.
j	Operation index of job J_i, $j \in \{1, \ldots, N_i\}$.
t	Machine type index, $t \in \{1, \ldots, N_T\}$.
k	Parallel machine index of type t, $k \in \{1, \ldots, N^M_t\}$.
s	Sublot index, $s \in \{1, \ldots, R_{i,j}\}$.

Parameters

N	Number of jobs.
B_i	Lot size of J_i.
N_i	Number of operations of J_i.
N_T	Number of machine types.
N^M_t	Number of parallel machines of type t.
$T^P_{i,j}$	Processing time of operation $O_{i,j}$.
$T^S_{i,j}$	Setup time of operation $O_{i,j}$.

Decision Variables

$R_{i,j}$	Number of sublots of operation $O_{i,j}$.
$H_{i,j,s}$	Size of sublot s of operation $O_{i,j}$.
$S^L_{i,j,s}$	The start time of sublot $L_{i,j,s}$.
$F^L_{i,j,s}$	The finish time of sublot $L_{i,j,s}$.
$S^O_{i,j}$	The start time of operation $O_{i,j}$.
$F^O_{i,j}$	The finish time of operation $O_{i,j}$.
$F^M_{t,k}$	The current finish time of machine $M_{t,k}$.
$X_{i,j,s,t,k}$	If sublot $L_{i,j,s}$ is processed on machine $M_{t,k}$, the value is 1; otherwise, the value is 0.
$C_{\max}$	The makespan of a solution.

The LHJSV aims to minimize the makespan. The computational model of LHJSV is described as follows:

$$\min \quad C_{\max} = \max \quad \left\{F^O_{i,j}\right\} \quad \forall i, j \tag{1}$$

subject to

$$1 \le R_{i,j} \le \min \quad \left(B_i, N^M_t\right), R_{i,j} \in \mathbb{Z}^+ \quad \forall i, j \tag{2}$$

$$1 \le H_{i,j,s} \le B_i, H_{i,j,s} \in \mathbb{Z}^+ \quad \forall i, j, s \tag{3}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

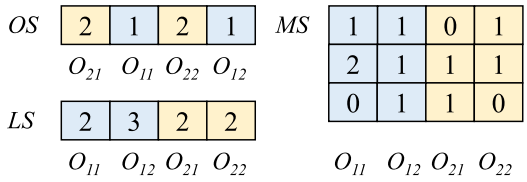


Fig. 2. Encoding method for LHJSV.

$$S^L_{i,j,s} = \max \quad \{F^M_{t,k} \times X_{i,j,s,k,t}, F^O_{i,j-1}\} \quad \forall i, j, s, k, t \tag{4}$$

$$F^L_{i,j,s} = S^L_{i,j,s} + T^S_{i,j} + n_s T^P_{i,j}/B_i \quad \forall i, j, s \tag{5}$$

$$F^M_{t,k} \geq F^L_{i,j,s} X_{i,j,s,k,t} \quad \forall i, j, s, k, t \tag{6}$$

$$S^O_{i,j} = \min_{s=1}^{n_s} \quad \{S^L_{i,j,s}\} \quad \forall i, j \tag{7}$$

$$F^O_{i,j} = \max_{s=1}^{n_s} \quad \{F^L_{i,j,s}\} \quad \forall i, j. \tag{8}$$

Eq. (1) represents the optimization objective. Constraints in (2) and (3) define the permissible range for the number of sublots and the size of each sublot for every operation. Eq. (4) determines the constrained earliest start time for each sublot. Eq. (5) calculates the finish time of each sublot. Eq. (6) evaluates the completion time of each parallel machine. Finally, (7) and (8) determine the start and finish times for each operation.

C. Vector-Based Encoding and Decoding

Encoding: In this work, a MA is adopted to address the LHJSV. The subproblems of LHJSV are encoded into three vectors: 1) OS; 2) LS; and 3) MS. Fig. 2 illustrates the encoding process for the solution depicted in Fig. 1. The LS and MS vectors follow the operation sequence from O_{11} to O_{22}. The size of the MS vector is $(\sum_{i=1}^{N} N_i) \times \max(N^M_t)$. The sublot size of $L_{i,j,s}$ is recorded in the kth row, where k corresponds to the machine index.

Decoding: First, the operation is retrieved from OS. The j_{th} occurrence of the number i then corresponds to the process $O_{i,j}$. Second, B_i lots of $O_{i,j}$ are split into $R_{i,j}$ sublots based on LS. Third, the sublot size is derived from the MS vector. Fourth, the start and finish times of each sublot are calculated based on (4) and (5). The finish time of the parallel machine is computed using (6). The start and finish times of each operation are calculated using (7) and (8). Finally, the makespan is obtained by (1).

D. Complexity of Solving LHJSV

LHJSV contains three subproblems. As for OS, there are N jobs, each job has N_i operations, and the total number of operations is $\sum_{i=1}^{N} N_i$. All operations of a job are encoded with the same number, and they are considered repeated items. Thus, the number of permutations is $Y = (\sum_{i=1}^{N} N_i)!/(\prod_{i=1}^{N} N_i!)$. As for LS and sublot MS, which can be regarded as B_i identical balls being placed into N^M_t identical boxes. Thus, the number of combinations for an operation $O_{i,j}$ is $Q_{i,j} = C(B_i + N^M_t - 1, N^M_t - 1)$. For all operations, the total number of combinations is $Z = \prod_{i=1}^{N} \prod_{j=1}^{N_i} Q_{i,j}$. Finally, the total number of exhaustions is $Y * Z$, which will grow extremely quickly as the dimension of the problem increases.

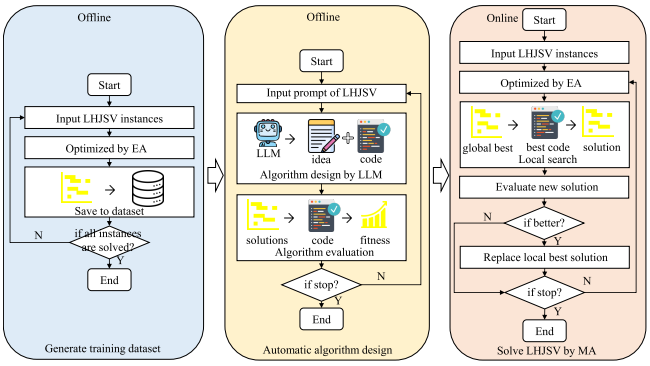


Fig. 3. Framework of LLMMA.

III. Our Approach: LLMMA

In this section, the motivation and details of designing LLMMA are introduced.

A. Framework of LLMMA

The framework of LLMMA is depicted in Fig. 3. Initially, an EA is employed to solve all LHJSV instances. The goal of utilizing LLM is to design the heuristic, which can enhance the exploitation of EA. Thus, the final best solutions obtained from this phase serve as the baseline for training the LLM to further improve the results of EA. Next, the LLM is utilized to design problem-specific heuristics through the proposed cooperative evolution framework. This enables the automatic generation of heuristics tailored to the complexities of LHJSV. Finally, the best-designed heuristics are integrated into the EA as a MA to solve LHJSV online. The reason that phase three is online is due to the fact that the order is usually inserted urgently. When a new order arrives, the LHJSV becomes a new case to solve. Additionally, the high timeliness of the algorithms is also a requirement for practical manufacturing, so the MA is set to be online, and a satisfactory solution is found in a short time.

B. Evolutionary Algorithm for LHJSV

Initialization: The initialization procedure is carried out in three steps. First, ps OS vectors are generated by repeating the job index $i \in \{1, \ldots, N\}$ for N_i times and then randomly permuting the sequence. Second, ps LS vectors are generated by randomly selecting a positive integer from the range $[1, \max(B_i, N^M_t)]$ for each operation. Third, based on the generated LS vector, the B_i lots are uniformly distributed into $R_{i,j}$ sublots, with each sublot randomly assigned to a parallel machine from its respective candidate machine set.

Crossover: In this study, we employ two types of crossover operators: 1) the precedence operation crossover (POX) [32] for OS and 2) the universal crossover (UX) [33] for LS and MS. The procedure is depicted in Fig. 4. The steps of the crossover process are as follows: 1) *Selection:* Two parent solutions are chosen: the current solution P_i, and another randomly selected from an elite mating pool, which is constructed using a two-player tournament selection strategy. 2) *POX:* The

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

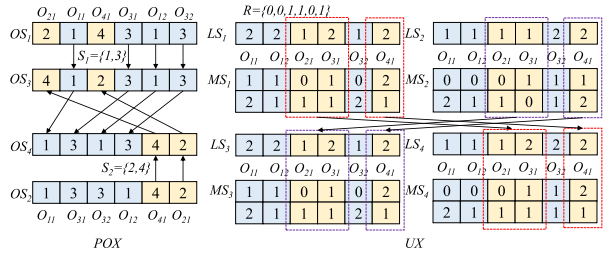


Fig. 4. Procedure of POX and UX operators.

job set is partitioned into two subsets. Operations belonging to each subset are preserved in their respective positions in the offspring. Operations not belonging to a subset are sequentially copied to the vacant positions of the other offspring. 3) *UX:* A random vector is generated. If the current index of the vector is equal to one, the LS and MS components of the two solutions are exchanged.

Mutation: Two simple mutation strategies are designed. As for OS, two genes are randomly exchanged with probability P_m. As for LS and MS, one operation is randomly selected and resplit the lots and reselects their machine with probability P_m. These strategies are adopted by all offspring.

Environmental Selection: The population and offspring are combined to compare their fitness. The solutions with the top ps smallest fitness are kept to the new population for the next generation.

C. LLM-Assisted Cooperative Evolution of Heuristic

Motivation: Previous work by [28] proposed a framework for solving simple combinatorial optimization problems, such as online bin packing and permutation flow shop scheduling. These tasks are relatively straightforward and can be easily described in a prompt. However, when dealing with more complex shop scheduling problems, such as the LHJSV, the logic and constraints are significantly more intricate. As a result, the prompt becomes long and complex, which can be difficult for LLMs to handle, potentially leading to the design of incorrect algorithms. Motivated by the "divide-and-conquer" approach, we propose a cooperative evolution framework for the automatic design of heuristics to tackle the challenges posed by complex scheduling problems.

Framework: The LHJSV problem is decomposed into two subproblems: 1) OS, and 2) LS and MS. To simplify the task and improve the success rate of designing a correct algorithm, we design two distinct types of prompts for each subproblem. These prompts reduce the complexity of the task, shorten the length of the input required for LLMs, and increase the likelihood of obtaining accurate solutions. The proposed framework is illustrated in Fig. 5, with further details presented in the following section.

Prompt: Prompt engineering plays a critical role in guiding a universal LLM to abstract expert knowledge from its prelearned data. The framework for prompt construction is based on the work of [28], which utilizes four core operators: $[E_1, E_2, M_1, M_2]$. These operators are described as follows: 1) E_1 generates a completely novel algorithm that differs significantly from its parent algorithms. 2) E_2 designs a new algorithm that is inspired by the parent algorithm's conceptual approach. 3) M_1 improves the current algorithm by refining its structure or performance. 4) M_2 modifies the parameters of the existing algorithm to enhance its functionality. All the operators in the framework proposed by [28] contain six key components, which are *task description, function name, input, output, information of input and output*, and *other relevant information*. However, as the complexity of the task increases, it becomes increasingly challenging for the LLM to generate feasible algorithms based solely on these components. To address this issue, we introduce a new element, *manual code*, which is added after the *task* description. This addition leverages the Chain-of-Thought reasoning ability of LLMs to help them comprehend the correct format and structure of the code necessary to solve the more complex problem. The components of the prompts for the two subproblems are illustrated in Figs. S-1 and S-2, in the supplementary materials.

Initialization: The algorithmic population is initialized by submitting the i_1 prompt to the LLM twice. The construction of the i_1 prompt follows a specific structure: *task description*, *manual code*. The prompt is formulated as follows: "First, describe your new algorithm and its main steps in one sentence. The description must be enclosed within braces. Next, implement the algorithm in Python as a function named *function name*. This function should accept *Input* and return *Output*. Additionally, provide *information on the input and output* and *other relevant information*." The populations for two tasks are initialized by embedding these components into the i_1 prompt and interacting with the LLM. Each solution consists of an algorithmic description, the corresponding code, and a fitness evaluation.

Algorithm evaluation: After initialization, the fitness of each solution is evaluated according to the following procedure. First, a baseline solution $S_i, i = 1, \ldots, N_C$ is generated by executing the EA on each LHJSV instance, where N_C is the number of instances. The stopping criterion for this process is set to MaxNFEs $= 2 \times 10^4$, with a population size of $ps = 100$, a crossover rate of $P_c = 1.0$, and a mutation rate of $P_m = 0.2$. Second, the OS vector OS of the solution S_i is input into its algorithmic population, while the LS and MS vectors are input into the populations of the other tasks. Each algorithm independently optimizes S_i for n_{ls} iterations. Third, a multiobjective indicator, similar to the fitness function used in SPEA2 [34], is employed to evaluate the performance of the algorithm. This indicator is formulated in (9). Notably, if the algorithm generated by LLM fails to run, the fitness is set to none, and the algorithm is abandoned in environmental selection first. If the algorithm runs successfully but with syntax errors and logical flaws, resulting in generating infeasible solutions, the solutions are repaired by the following steps. First, the infeasible part of these solutions is found. Next, the unscheduled operations are rescheduled with random permutation, and are inserted into the same empty space of the solution. Finally, the infeasible MS is reallocated from the candidate set

$$F_j = \sum_{i=1}^{N_C} \phi_i - \frac{1}{RN_C} \sum_{t=1}^{R} \sum_{i=1}^{N_C} \frac{C_{\max}(A_j) - C_{\max}(S_i)}{C_{\max}(S_i)} \quad \forall j. \quad (9)$$

The first component is the number of instances in which the LLM-designed algorithm achieves a better performance

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

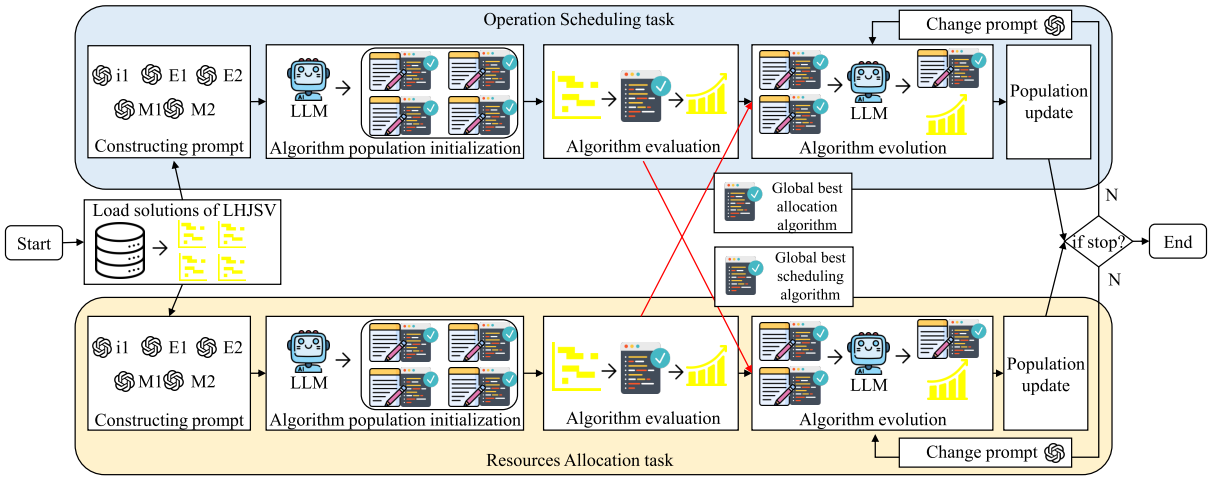


Fig. 5. Framework of CEoH.

gap. The second component is the average relative percentage deviation (*RPD*) for each instance. Specifically, if $\text{RPD}_i < 0$ for $i = 1, \ldots, N_C$, then $\phi_i = 1$; otherwise, $\phi_i = 0$. A higher value of F_j indicates better performance for the algorithm A_j. The rationale behind the design of (9) is to identify algorithms that not only achieve a better average RPD across all instances but also improve performance on each individual instance. Regarding initialization, the algorithmic fitness is computed solely by executing its corresponding code.

Algorithm co-evolution: Drawing inspiration from the "divide-and-conquer" strategy, the LHJSV problem is decomposed into two algorithmic populations. This decomposition simplifies the complexity of the prompt and aids the LLM in understanding the LHJSV problem more effectively. The ultimate objective is to design a comprehensive algorithm that encapsulates expert knowledge and can optimize all subproblems of the LHJSV simultaneously. To achieve this, both tasks must evaluate their respective algorithms cooperatively. The process unfolds in two main steps: First, the best algorithm for each task is identified. Second, the new algorithm for each task is generated by submitting the set of operators $[E_1, E_2, M_1, M_2]$ to the LLM. Detailed information on these prompts is provided in the supplementary material. Finally, the fitness of each algorithm is evaluated by cooperatively optimizing a solution S_i using the best solution from the other task. The algorithm co-evolution is terminated for A_g generations.

Algorithmic Population Update: Each algorithmic population is updated by preserving the top maximum A_p algorithms, where A_p is the size of algorithmic population.

Algorithm 1 LLMMA

Input: population size ps, number of local search n_{ls}, mutation rate P_m, global best algorithm of scheduling A_{sc}, global best algorithm of allocation A_{al}, maximum number of function evaluation $MaxNFEs$
Output: Global best solution g_{best}
$P_0 \leftarrow$ Initialization(ps).
$F_0 \leftarrow$ Decoding(P_0).
$g_{best} \leftarrow$ argmin(P_0, F_0).
$NFEs \leftarrow NFEs + ps$
while $NFEs <= MaxNFEs$ **do**
  $M_t \leftarrow$Tournament selection(ps,P_t)
  $C_t \leftarrow$Crossover(P_t, M_t, ps)
  $Q_t \leftarrow$Mutation(C_t, P_m, ps)
  $F_t^C \leftarrow$ Decoding(Q_t).
  $NFEs \leftarrow NFEs + 2 * ps$
  $[P_{t+1}, F_{t+1}] \leftarrow$Environmental selection($ps$,$P_t$, Q_t)
  $g_{best} \leftarrow$argmin(P_{t+1}, F_{t+1})
  for $i = 1$ *to* n_{ls} **do**
    $OS' \leftarrow A_{sc}(g_{best}(OS))$
    $[LS', MS'] \leftarrow A_{al}(g_{best}(LS), g_{best}(MS))$
    $F' \leftarrow$ Decoding(OS', LS', MS').
    $NFEs \leftarrow NFEs + 1$
    $g_{best} \leftarrow$ argmin(g_{best}, $[OS', LS', MS']$)

D. LLM-Assisted Memetic Algorithm

The prevailing algorithmic framework for solving complex shop scheduling problems is based on memetic computing, which combines an EA with several problem-specific local search operators. While the EA serves as a generalized framework that is straightforward to implement, the most computationally demanding aspect lies in the development of an efficient knowledge-based local search. Consequently, the heuristic is automatically designed by the LLM and adopted as a local search operator to enhance the exploitation of EA. Following the offline design of an algorithm tailored for the LHJSV problem, the LLM identifies and refines the knowledge necessary to optimize the complex LHJSV. Each knowledge reflects an idea of heuristic and is implemented by LLM to the code. The code of the algorithm with the best fitness is subsequently integrated into the EA, as outlined in Section III-B. Ultimately, a MA is derived with the knowledge-based heuristic, which is capable of solving the LHJSV problem online. The complete procedure of the LLMMA is detailed in Algorithm 1.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

IV. Experimental Results

In this section, the hyperparameters of the proposed LLMMA are analyzed. Next, an ablation experiment of LLMMA is designed to assess the effectiveness of each improvement. Finally, a comparison experiment is designed to validate the superior performance in solving LHJSV. All experiments were conducted on a 13th Gen Intel Core i9-13900K 3.00 GHz CPU, with the code implemented in Python 3.8, CUDA 12.1, and PyTorch 2.1.1.

A. Dataset and Metrics

The dataset for LHJSV is based on the JSP benchmark TA01-80 [35], which can be obtained from the link.[1] Twenty instances are uniformly selected from TA. The lot size B_i for each job is randomly set to 5 or 10. The number of parallel machines for each type of machine is randomly set to 2 or 3. The setup time for each operation is randomly sampled from the range [5,10]. The metric for measuring the training of the LLM is (9), and the metric for solving LHJSV is RPD, where $\text{RPD} = (C_{\max}(A_j) - LB)/LB$ and $LB = \min(C_{\max}(A_j)) \quad \forall j$. The training baseline for the LLM is established based on the optimal outcomes achieved across 20 runs, with each instance being solved by EA.

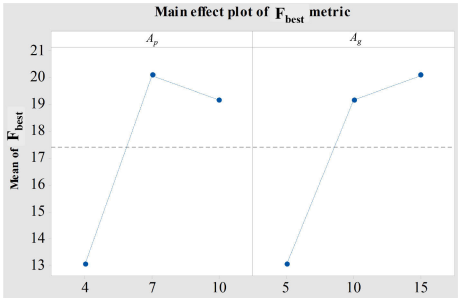


Fig. 6. Main effect plot of F_{best}.

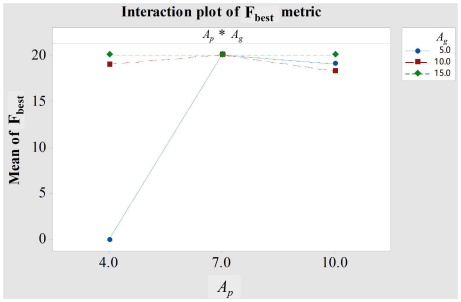


Fig. 7. Factor interaction plot of F_{best}.

B. Hyperparameter Analysis

To identify the optimal parameter configuration for LLMMA, three hyperparameter experiments were designed.

The first experiment investigates the cooperative evolution of heuristics (CEoHs), focusing on two key parameters: 1) the algorithmic population size, denoted as A_p, and 2) the number of generations, denoted as A_g. A full factorial design with two factors and three levels is employed to explore the best training parameters for CEoH. Specifically, the values of A_p are chosen from the set $\{4, 7, 10\}$, while A_g takes values from the set $\{5, 10, 15\}$. Since the training of CEoH is conducted offline, the total number of evaluations is not constrained. However, considering the computational cost associated with training the LLM, the GPT-4o-mini LLM API is selected for this experiment. Each algorithm is evaluated by independently solving the baseline S_i five times, and the best metric, denoted as F_{best} in (9), is recorded as the performance indicator. The results of the experiment are shown in Figs. 6 and 7. The F_{best} value serves as an indicator of improvement, with higher values reflecting better performance of the LLM-designed algorithm. The optimal value for A_p is found to be 7, while no clear preference emerges for A_g. Based on the analysis in Fig. 7, no significant differences are observed when $A_p = 7$, suggesting that A_g should be set to 15, assuming the training cost is not a limiting factor.

The second experiment focuses on the evaluation of three critical parameters in the MA: population size ps, number of local searches n_{ls}, and mutation rate P_m. A full factorial design with three factors, each at three levels, is employed to analyze the optimal parameter configuration for the MA. The parameter values considered are $ps = \{20, 50, 100\}$, $n_{ls} = \{1, 5, 10\}$, and $P_m = \{0.1, 0.2, 0.3\}$. The algorithm, designed by LLM, is trained with $A_p = 4$, $A_g = 5$, and the GPT-4o-mini model. Each configuration of the MA is executed 20 times independently, with the maximum number of function evaluations set to $\text{MaxNFEs} = 2 \times 10^4$. To accommodate the time-intensive nature of the Python-based computations, the evolution process is parallelized across ten processes. The average relative percentage deviation, denoted as RPD_{aver}, is used as the performance indicator. The results of the factor analysis are presented in Figs. 8 and 9. A lower value of RPD_{aver} corresponds to better performance of the MA. Based on the analysis in Fig. 8, it is clear that the number of local searches, n_{ls}, exhibits a distinct knee point, with the optimal value for n_{ls} being 5. From Fig. 9, it is observed that, when $n_{ls} = 5$, the best values for ps and P_m are 20 and 0.3, respectively.

Third, after identifying the optimal training parameters for CEoH and the best configuration for the MA, the effectiveness of different LLM APIs is evaluated. The selected models for comparison include GPT-4o-mini (OpenAI), GPT-4o (OpenAI), and Claude-3.5-Sonnet-20241022 (Anthropic). Each model is trained with the following parameters: $A_p = 7$ and $A_g = 5$. The local search algorithm, designed by different LLMs, is tested using the parameters $ps = 20$, $n_{ls} = 5$, and $P_m = 0.3$. The performance is assessed through the training metric F_{best}, the test metric RPD_{aver}, the training cost, and the quality-price ratio. The results are presented in Table II. The GPT-4o model demonstrates the best performance in both training and testing, indicating its robust capability in understanding the complexities of the LHJSV and designing efficient algorithms. On the other hand, GPT-4o-mini proves to be the most cost-effective

[1] https://github.com/tamy0612/JSPLIB

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

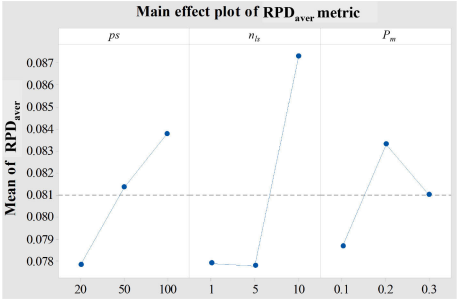


Fig. 8. Main effect plot of RPD_{aver}.

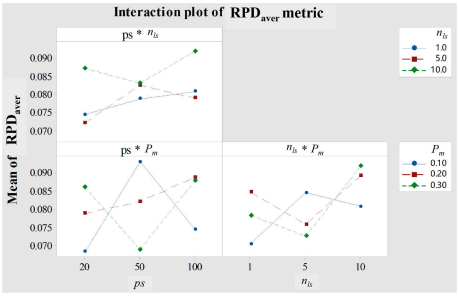


Fig. 9. Factor interaction plot of RPD_{aver}.

TABLE II
COMPARISON RESULTS OF DIFFERENT LLM APIS

LLM	F_{best}	RPD_{aver}	Cost	ratio
gpt-4o-mini	19.05527	0.049591	0.9676$	**0.05125**
gpt-4o	**20.07229**	**0.040468**	3.5588$	0.011371
claude-3.5-sonnet	2.9967	0.107258	5.2045$	0.020609

F_{best} (max), RPD_{aver} (min), Cost (min), ratio (max)

model for training. Conversely, Claude-3.5-Sonnet fails to handle the complexity of scheduling problems effectively. Thus, GPT-4o and GPT-4o-mini are found to be suitable choices for designing algorithms for complex shop scheduling problems. Additionally, other LLM models, including SparkDesk, SparkDesk-v3.5, Moonshot-v1-128k, Deepseek-Coder, Deepseek-Chat, ERNIE-4.0-8k, GPT-3.5-Turbo, and Qwen-Turbo, were also tested. However, these models were unable to design feasible algorithms for the coupled LS and MS tasks.

The detailed test results of various LLM APIs are presented in Table III. The algorithms generated by different LLMs are denoted as LLMMA1, LLMMA2, and LLMMA3. Upon analysis, it is observed that LLMMA2 outperforms the other algorithms, demonstrating its superior ability to design an algorithm tailored to the LHJSV problem. The best algorithms derived from GPT-4o are shown in Fig. S-3, in the supplementary materials, while the best algorithms from GPT-4o-mini are displayed in Fig. fig:S-4, in the supplementary materials. In terms of OS scheduling, the LLM introduces a random shift of one step for all operations of a randomly selected job. Although the LLM does not fully comprehend the intricate encoding of the OS, its algorithm compensates by generating large search steps, potentially uncovering better solutions. For the LS and MS task, the LLM aims to balance

TABLE III
RESULTS OF RPD_{aver} FOR LLMMA WITH DIFFERENT LLM APIS

Instance	N-N_i-N_T	LB	LLMMA1	LLMMA2	LLMMA3
LHJSV01	15-15-15	808	0.057054	**0.030817**	0.114109
LHJSV02	15-15-15	823	**0.059174**	0.061847	0.104253
LHJSV03	15-15-15	784	0.076148	**0.047194**	0.142347
LHJSV04	20-15-15	946	**0.040063**	0.049577	0.083615
LHJSV05	20-15-15	984	0.045935	**0.035874**	0.116972
LHJSV06	20-20-20	1238	0.076575	**0.060016**	0.115913
LHJSV07	20-20-20	1171	0.040307	**0.032109**	0.133732
LHJSV08	20-20-20	1144	0.064161	**0.028234**	0.152622
LHJSV09	30-15-15	1354	**0.051994**	0.058419	0.108936
LHJSV10	30-15-15	1268	**0.052918**	0.058833	0.104101
LHJSV11	30-20-20	1634	0.030477	**0.020135**	0.084945
LHJSV12	30-20-20	1431	0.073795	**0.064081**	0.166737
LHJSV13	30-20-20	1529	**0.033486**	0.03571	0.097973
LHJSV14	50-15-15	1878	0.0418	**0.035037**	0.099042
LHJSV15	50-15-15	1966	0.025178	**0.016582**	0.066887
LHJSV16	50-20-20	2185	0.056156	**0.04357**	0.099085
LHJSV17	50-20-20	2107	0.044993	**0.025866**	0.097342
LHJSV18	50-20-20	2245	0.05029	**0.049265**	0.108419
LHJSV19	100-20-20	3896	0.042685	**0.035318**	0.0827
LHJSV20	100-20-20	3841	0.028638	**0.02088**	0.065426
aver			0.049591	**0.040468**	0.107258

LLMMA1 (gpt-4o-mini), LLMMA2 (gpt-4o), LLMMA3(claude-3.5-sonnet)

the machine load by assigning sublots to the parallel machine with the minimum load, and the generated code is correct. This balanced LS approach ensures optimal utilization of parallel machines, ultimately reducing the makespan. These results confirm that the designed prompt successfully enables the LLM to abstract the necessary knowledge for solving the LHJSV problem, validating the effectiveness of our approach.

Fourth, the influence of the training baseline is tested. Five local optimal solutions are obtained from different generations of EA, including 10%, 30%, 50%, 70%, and 100% of MaxNFEs. The LLM's population is 7, the algorithmic iteration is 5, and the API is GPT-4o-mini. After finishing designing the algorithm, the algorithm with the best F_{best} is embedded in EA. The variant algorithms are named LLMMA10-LLMMA100, indicating the baseline obtained from different generations of EA. The results are shown in Table. S-I in the supplementary materials. The LLMMA100 obtains the best RPD performance compared to other baselines, indicating the LLM can design the enhanced algorithm if the baseline is the best solution for EA. The other LLMMAs cannot obtain better performance than GA because the LLM can get a better solution than its baseline, which is not the optimal solution of EA. However, the designed algorithm finds a better solution than the final optimal solution of EA, and the designed algorithm will consume additional computational resources, reducing the times of crossover. Thus, the performance of LLMMA10-70 is worse than GA.

C. Ablation Experiment

To validate the effectiveness of the proposed improvements, ablation experiments were conducted on both the CEoH and MA. Due to the cost associated with GPT-4o, GPT-4o-mini was selected for algorithm design in this experiment. The training parameters used were $A_p = 7$ and $A_g = 5$, while the test parameters were $ps = 20$, $n_{ls} = 5$, and $P_m = 0.3$. In cases where the LLM designed infeasible algorithms, the metric was set to $F = -100$.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

The ablation study proceeded as follows: First, two prompts related to the LHJSV problem were combined into a single, complex prompt, and the LLM generated an ensemble algorithm in a single interaction. The resulting validated algorithm was named CEoH1. Second, to analyze the impact of the Chain-of-Thought mechanism, the manual code was removed from the prompt, yielding the variant algorithm CEoH2. Third, the LLM-designed local search component was removed from the MA to assess its effectiveness and determine if the LLM could abstract expert knowledge through the designed prompt. The resulting variant algorithm was named EA. Each of the algorithms was executed 20 times, with the stopping criterion set to MaxNFEs $= 2 * 10^4$. The performance metrics, including those defined in (9) and the RPD, were collected for further analysis.

The results of the training performance for the variant algorithms are as follows: $F_{\text{best}}(\text{CEoH1}) = 15.01831$, $F_{\text{best}}(\text{CEoH2}) = 15.03009$, and $F_{\text{best}}(\text{CEoH}) = 19.05527$. These metrics indicate that both CEoH1 and CEoH2 achieved better results across 15 instances, while the full CEoH algorithm improved the baseline solutions across 19 instances. Specifically, CEoH1 improved the average RPD by 1.831%, CEoH2 by 3.009%, and the full CEoH algorithm achieved an improvement of 5.527%. The training results suggest that removing the Chain-of-Thought mechanism and the CEoHs significantly reduces the comprehensibility of the LLM, which in turn hinders the LLM's ability to design efficient algorithms quickly.

The test results for RPD across all variant MAs are shown in Table IV. Each MA utilized the LLM-designed algorithm with the best F metric. The algorithms embedded in the MAs designed by variant prompt CEoH1, CEoH2, or CEoH are named MA-CEoH1, MA-CEoH2, and MA-CEoH, respectively. Based on the results, the MA-CEoH algorithm achieved the best RPD_{aver} across 15 instances and reached the best solution in 12 instances, further indicating that removing the Chain-of-Thought mechanism and the CEoHs results in the LLM designing inefficient algorithms. Additionally, the MA-CEoH algorithm achieved the best average RPD_{best} and RPD_{aver} across all instances, demonstrating that the proposed CEoH approach yields the best overall performance. This outcome validates the effectiveness of the improvement strategy. When the cooperative evolution was replaced with a single population and a unified prompt, the RPD_{aver} increased by 1.75%. This increase can be attributed to the LLM's inability to process complex, lengthy prompts, leading to a higher rate of infeasible algorithm generation. On the other hand, removing the Chain-of-Thought, which deletes the manual encoding caused the RPD_{aver} to increase by 1.35%. This suggests that the LLM struggles to understand complex shop scheduling problems without the guidance provided by the manually encoded code, which acts as a form of instructional imitation.

TABLE IV
STATISTICAL RESULTS OF RPD (MIN) METRIC FOR VARIANT ALGORITHMS

		GA		MA-CEoH1		MA-CEoH2		MA-CEoH	
Instances	LB	best	mean	best	mean	best	mean	best	mean
LHJSV01	799	0.1690	0.2222	0.0263	0.0820	**0.0000**	**0.0678**	0.0163	0.0690
LHJSV02	823	0.1203	0.1859	0.0267	0.0770	0.0134	**0.0532**	**0.0000**	0.0592
LHJSV03	795	0.1799	0.2177	**0.0000**	0.0717	0.0289	0.0757	0.0164	**0.0613**
LHJSV04	922	0.1714	0.1948	**0.0000**	0.0753	0.0412	0.0824	0.0260	**0.0671**
LHJSV05	995	0.0965	0.1439	0.0050	0.0430	**0.0000**	**0.0302**	0.0060	0.0344
LHJSV06	1268	0.1530	0.1841	**0.0000**	**0.0409**	0.0134	0.0421	0.0024	0.0511
LHJSV07	1186	0.1636	0.2062	0.0093	0.0331	0.0160	0.0528	**0.0000**	**0.0272**
LHJSV08	1154	0.1638	0.2475	0.0078	0.0604	0.0208	0.0673	**0.0000**	**0.0549**
LHJSV09	1354	0.1204	0.1810	0.0037	0.0631	0.0465	0.0827	**0.0000**	**0.0520**
LHJSV10	1268	0.1215	0.1718	0.0347	0.0823	0.0457	0.0621	**0.0000**	**0.0529**
LHJSV11	1606	0.1276	0.1804	**0.0000**	**0.0384**	0.0187	0.0574	0.0174	0.0484
LHJSV12	1471	0.1890	0.2262	0.0557	0.0789	0.0197	0.0641	**0.0000**	**0.0446**
LHJSV13	1507	0.1393	0.1982	0.0239	0.0690	**0.0000**	0.0676	0.0179	**0.0486**
LHJSV14	1884	0.0632	0.1190	0.0356	0.0717	0.0212	0.0562	**0.0000**	**0.0385**
LHJSV15	1966	0.0651	0.1294	0.0417	0.0637	0.0132	0.0478	**0.0000**	**0.0252**
LHJSV16	2204	0.1615	0.2054	0.0336	0.0679	0.0123	0.0659	**0.0000**	**0.0471**
LHJSV17	2127	0.1387	0.1791	0.0141	0.0528	**0.0000**	0.0452	0.0085	**0.0352**
LHJSV18	2245	0.0989	0.1659	0.0307	0.0749	0.0254	0.0582	**0.0000**	**0.0503**
LHJSV19	3908	0.0875	0.1338	0.0591	0.0771	0.0402	0.0663	**0.0000**	**0.0395**
LHJSV20	3841	0.0885	0.1098	0.0424	0.0616	0.0289	0.0590	**0.0000**	**0.0286**
aver		0.1309	0.1801	0.0225	0.0642	0.0203	0.0602	**0.0055**	**0.0467**

D. Comparison Experiment

To verify the effectiveness of the proposed LLMMA in solving LHJSV, a comparative analysis is conducted against several existing algorithms, including iterative search algorithms (HHIGA [14] and RLABC [36]), a generative scheduling algorithm (HGNN [16]), and an LLM-based algorithm (EoH [28]). The primary challenge in this work lies in optimizing LS and MS for sublots. However, both HHIGA and RLABC employ fixed rules, such as equal or consistent sublot sizes. To address this limitation, the rules are replaced by the novel crossover and mutation operators introduced in this study. While HGNN is not an exact fit for the LHJSV, it is the only deep RL algorithm in current research. To adapt HGNN for LHJSV, an additional agent is introduced to determine the lot-streaming rule for each operation, enabling the transformation of LHJSV into a flexible job shop scheduling problem (FJSP). In this representation, the machines in the candidate set for each operation are assumed to have identical processing times, and the number of sublots is constrained by the maximum batch size, as specified in [16]. EoH, as an algorithmic design framework, uses a unified prompt that integrates all subproblems of LHJSV. The LLM, utilizing GPT-4o-mini, generates the best algorithm, which is then embedded into the MA for local search optimization.

The population size ps for both HHIGA and RLABC is set to 100. In the case of RLABC, the probability of crossover with the global best is set to $PE = 0.7$, the rate of scouter bees is $P_S = 0.3$, the roulette iteration is 1500, and the number of local searches is 10. For HGNN, the training epoch is 1000, with 20 iterations for generating new cases and ten iterations for validation. The training and validation cases consist of ten jobs and five operations, with a greedy selection rule for sampling distributions. Additional training parameters can be found in the supplementary link,[2] as HGNN is an extension of their work. The training and testing parameters for EoH follow the optimal settings discussed in Section IV-B.

First, the performance of the GA, LLMMA1 (GPT-4o-mini), and LLMMA2 (GPT-4o) is compared with state-of-the-art algorithms on the benchmark described in Section IV-A. The

[2]https://github.com/songwenas12/fjsp-drl

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS OF RPD FOR COMPARISON ALGORITHMS ON 20 INSTANCES

Instance	LB	GA		HHIGA		RLABC		DRL-G		MA-EoH		LLMMA1		LLMMA2	
		best	mean	best	mean	best	mean	best	mean	best	mean	best	mean	best	mean
LHJSV01	808	0.1559	0.2085	0.0483	0.0905	0.5693	0.6677	0.7871	1.0347	0.0012	0.0687	0.0050	0.0571	**0.0000**	**0.0308**
LHJSV02	823	0.1203	0.1859	0.0231	**0.0547**	0.5954	0.6520	0.7230	0.9744	0.0304	0.0789	**0.0000**	0.0592	0.0049	0.0618
LHJSV03	784	0.1964	0.2348	0.0842	0.1214	0.6224	0.7189	0.9273	1.0691	0.0383	0.0819	0.0306	0.0761	**0.0000**	**0.0472**
LHJSV04	946	0.1416	0.1645	0.0740	0.1262	0.5803	0.6365	0.8647	0.9900	0.0359	0.0666	**0.0000**	**0.0401**	0.0180	0.0496
LHJSV05	978	0.1155	0.1638	0.0409	0.0740	0.5808	0.6484	0.7035	0.9241	**0.0000**	0.0540	0.0235	0.0524	0.0061	**0.0422**
LHJSV06	1238	0.1809	0.2128	0.0711	0.1146	0.6341	0.7009	0.7908	1.0103	0.0242	0.0691	0.0267	0.0766	**0.0000**	**0.0600**
LHJSV07	1171	0.1785	0.2217	0.0615	0.1142	0.4927	0.6237	0.7071	0.8599	0.0325	0.0538	0.0128	0.0403	**0.0000**	**0.0321**
LHJSV08	1144	0.1740	0.2584	0.1250	0.1401	0.6486	0.7026	0.7220	1.0041	0.0114	0.0590	0.0087	0.0642	**0.0000**	**0.0282**
LHJSV09	1354	0.1204	0.1810	0.0554	0.0854	0.5495	0.6219	0.7600	0.9773	0.0103	0.0839	**0.0000**	**0.0520**	0.0170	0.0584
LHJSV10	1268	0.1215	0.1718	0.0718	0.0983	0.6065	0.6610	0.7744	0.8721	0.0410	0.0695	**0.0000**	**0.0529**	0.0174	0.0588
LHJSV11	1634	0.1083	0.1602	0.0141	0.0573	0.5214	0.5947	0.5796	0.7361	0.0067	0.0397	**0.0000**	0.0305	0.0086	**0.0201**
LHJSV12	1431	0.2222	0.2604	0.0657	0.1094	0.7009	0.7496	0.8477	0.9645	0.0419	0.0832	0.0280	0.0738	**0.0000**	**0.0641**
LHJSV13	1525	0.1259	0.1841	0.0223	0.0531	0.5266	0.5877	0.5469	0.8445	**0.0000**	0.0558	0.0059	**0.0362**	0.0026	0.0384
LHJSV14	1878	0.0666	0.1226	0.0330	0.1103	0.5032	0.5450	0.5740	0.8081	0.0474	0.0807	0.0032	0.0418	**0.0000**	**0.0350**
LHJSV15	1966	0.0651	0.1294	0.0590	0.0805	0.4573	0.4981	0.4695	0.7198	0.0346	0.0614	**0.0000**	0.0252	0.0031	**0.0166**
LHJSV16	2185	0.1716	0.2159	0.0989	0.1527	0.5350	0.5719	0.6650	0.7885	0.0568	0.0859	0.0087	0.0562	**0.0000**	**0.0436**
LHJSV17	2107	0.1495	0.1903	0.0645	0.0918	0.5501	0.5814	0.6578	0.7564	0.0204	0.0528	0.0180	0.0450	**0.0000**	**0.0259**
LHJSV18	2245	0.0989	0.1659	0.0459	0.0797	0.5069	0.5581	0.6134	0.7742	0.0463	0.0710	**0.0000**	0.0503	0.0267	**0.0493**
LHJSV19	3896	0.0909	0.1373	0.0644	0.1084	0.3729	0.4249	0.5465	0.6050	0.0608	0.0718	0.0031	0.0427	**0.0000**	**0.0353**
LHJSV20	3841	0.0885	0.1098	0.0953	0.1051	0.3918	0.4110	0.4543	0.5668	0.0364	0.0603	**0.0000**	0.0286	0.0008	**0.0209**
aver		0.1346	0.1840	0.0609	0.0984	0.5473	0.6078	0.6857	0.8640	0.0288	0.0674	0.0087	0.0500	**0.0053**	**0.0409**

population size for GA is set to 20. Each algorithm is executed 20 times independently, with the stopping criterion set to MaxNFEs $= 2 \times 10^4$. The statistical results of the RPD metric are summarized in Table V. Upon analysis, LLMMA2 achieves the best average RPD_{aver} across 15 instances and attains the best solution in ten instances. LLMMA1 outperforms the state-of-the-art algorithms in terms of RPD_{aver} on 17 instances and achieves a superior RPD_{best} on 16 instances, outperforming all algorithms except LLMMA2. Specifically, compared to GA, HHIGA, RLABC, DRL-G, and MA-EoH, LLMMA1 reduces the average RPD_{aver} by 13.4%, 4.84%, 55.78%, 81.4%, and 1.74%, respectively. The runtime of all algorithms is recorded in Table VII. As all algorithms are written in Python, the iterative algorithms take longer to execute. DRL-G runs the fastest due to the rapid inference of the end-to-end model. Compared to GA, LLMMA requires twice as much time due to the added overhead of the local search. LLMMA2 is more time-consuming than LLMMA1 because GPT-4o has a more complex function. MA-EoH is slower than LLMMA1 because its prompt integrates all subproblems, which makes it more complex.

RLABC performs poorly due to its restart strategy, which reinitializes elite solutions to escape local optima. DRL-G also underperforms because the heterogeneous graph neural network fails to adequately capture the complexities of LHJSV, and the large policy space hinders the discovery of useful knowledge. This highlights the need for a problem-specific model and training methodology for LHJSV. Additionally, the performance of EoH is suboptimal because the single-population framework struggles with handling long prompts, making it challenging for the LLM to design effective and correct algorithms for complex shop scheduling problems. When the LLM is upgraded to GPT-4o, the performance of the compared algorithms improves significantly: by 14.31%, 5.75%, 56.69%, 82.31%, and 2.65% over GA, HHIGA, RLABC, DRL-G, and MA-EoH, respectively. The results of the Wilcoxon test are presented in Table VI, confirming that LLMMA1 is significantly better than all algorithms except LLMMA2. This highlights the effectiveness of the proposed LLMMA. The success of LLMMA1 can be attributed to the prompt design, which incorporates both the Chain-of-Thought and cooperative evolution frameworks. When GPT-4o is employed in LLMMA, LLMMA2 demonstrates a statistically significant improvement, outperforming all other algorithms.

TABLE VI
RANKS COMPUTED BY THE WILCOXON TEST ON DATASET

	(1)	(2)	(3)	(4)	(5)	(6)	(7)
GA (1)	-	0.0	210.0	210.0	0.0	0.0	0.0
HHIGA (2)	210.0	-	210.0	210.0	10.0	1.0	1.0
RLABC (3)	0.0	0.0	-	210.0	0.0	0.0	0.0
DRL-G (4)	0.0	0.0	0.0	-	0.0	0.0	0.0
MA-EoH (5)	210.0	200.0	210.0	210.0	-	6.0	0.0
LLMMA1 (6)	210.0	209.0	210.0	210.0	204.0	-	25.0
LLMMA2 (7)	210.0	209.0	210.0	210.0	210.0	185.0	-

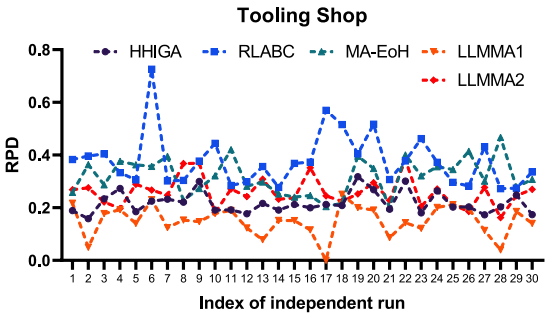


Fig. 10. Results of compared algorithms on the real-world case.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
RESULTS OF RUN TIME (SECOND) FOR COMPARISON ALGORITHMS ON 20 INSTANCES

Instances	GA	HHIGA	RLABC	DRL-G	MA-EoH	LLMMA1	LLMMA2
LHJSV01	14.58	14.19	96.30	**2.12**	39.18	31.07	42.34
LHJSV02	14.51	13.57	94.86	**2.10**	38.74	31.13	40.61
LHJSV03	14.73	13.86	96.39	**2.10**	39.34	31.90	41.29
LHJSV04	18.70	18.56	127.33	**2.79**	54.37	40.36	53.92
LHJSV05	18.76	18.51	128.16	**2.80**	60.25	40.30	52.93
LHJSV06	23.23	23.56	166.68	**3.74**	66.49	50.31	71.12
LHJSV07	23.52	23.95	167.46	**3.74**	66.31	52.68	65.74
LHJSV08	23.66	23.97	168.70	**3.74**	75.14	52.38	68.85
LHJSV09	26.14	26.96	188.19	**4.21**	83.47	57.80	77.71
LHJSV10	26.42	27.33	191.04	**4.22**	73.78	59.07	77.50
LHJSV11	33.98	35.23	251.39	**5.64**	99.01	75.79	103.74
LHJSV12	35.59	35.55	253.90	**5.69**	106.67	77.11	103.54
LHJSV13	38.12	35.32	251.48	**5.70**	107.40	77.08	101.14
LHJSV14	47.97	44.21	313.27	**7.12**	116.31	95.03	125.78
LHJSV15	48.04	44.74	315.18	**7.11**	120.54	95.68	129.14
LHJSV16	62.66	59.32	418.47	**9.53**	164.97	125.59	172.51
LHJSV17	63.14	59.62	420.71	**9.89**	175.49	126.34	173.85
LHJSV18	62.93	59.89	418.58	**9.77**	156.99	126.36	172.50
LHJSV19	128.33	125.85	836.45	**27.42**	336.26	254.90	332.91
LHJSV20	128.65	126.71	840.70	**27.23**	308.75	257.90	315.55

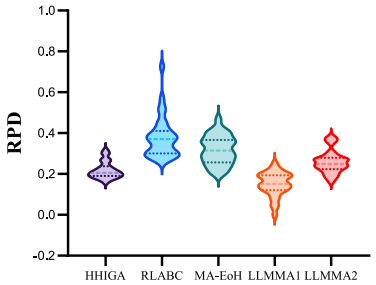


Fig. 11. Violin plot of RPD on the real-world case.

Second, all algorithms are tested on a real-world case from the tooling shop of the aircraft manufacturing enterprise, which mainly produces brackets, connecting lugs, angle lugs, connectors, protective covers, and other products. This case contains 23 jobs and a total has 575 operations and 16 types of machines. The detailed information is introduced in supplementary files. HHIGA, RLABC, MA-EoH, LLMM1, and LLMMA2 are independently executed for 30 times. Due to the different operation numbers of each job, the algorithm designed by LLM is manually modified to ensure feasibility but not on the idea itself. The results of RPD are shown in Fig. 10 and the violin plot is shown in Fig. 11. The LLMMA1 obtains better performance than other algorithms, indicating its idea is suitable for the case of jobs with different operation numbers. Because it considers the cost of processing time and setup time of each operation. The more cost, the more number of sublot for an operation. However, LLMMA2 only considers fine-tuning the sublot size according to machine load, which is not effective in the case of variant operation numbers.

V. CONCLUSION

In this study, we investigate a lot-streaming hybrid job shop scheduling problem with variant sublots. To automatically discover problem-specific knowledge and design the corresponding algorithm, we propose a MA assisted by a LLM. To facilitate the LLM's understanding of complex problems, we incorporate code into the initial prompt, leveraging the Chain-of-Thought framework. Furthermore, a CEoHs framework is designed to streamline the LHJSV prompt, thereby enhancing the feasibility of generating effective algorithms. The results of the ablation experiments demonstrate that the proposed improvements yield better training metrics and reduce the average RPD by 1.35% and 1.75% on the test dataset. Hyperparameter optimization results reveal that GPT-4o-mini offers the most cost-effective solution, while GPT-4o delivers the highest performance. Comparative experiments on the benchmark show that the proposed LLMMA outperforms existing algorithms on 85% of instances, improving the average RPD_{aver} by 13.4%, 4.84%, 55.78%, 81.4%, and 1.74%, relative to GA, HHIGA, RLABC, HGNN, and EoH, respectively. Additionally, experimental results on a real-world case confirm that the proposed method significantly outperforms other algorithms. From the LLM-designed algorithm, the following insights can be derived: 1) When the setup time is much smaller than the processing time, dividing the lot into as many batches as possible accelerates production. 2) When the gap between setup time and processing time is minimal, dividing more sublots for more resource-intensive processes leads to faster production. In conclusion, the cooperative automatic algorithm design framework proposed in this work can serve as a general methodology for solving other complex shop scheduling problems. When facing a new shop scheduling problem, it can be divided into several subproblems. Each subproblem can generate an algorithmic population. Then, all populations can adopt the proposed framework for designing heuristics based on problem-specific prompts. Finally, the designed heuristic is embedded into EA to generate a MA.

In future research, the following directions will be explored: 1) Enhancing the LLM's understanding of more complex constraints in real-world manufacturing environments. 2) Utilizing LLMs to design RL-based models for hyperparameter optimization and hyperheuristics. 3) Developing methods to efficiently design algorithms with reduced training costs.

REFERENCES

[1] (Airbus, Blagnac, France). *2024 Global Market Forecast*. 2024, Accessed: Mar. 4, 2025. [Online]. Available: https://www.airbus.com/en/products-services/commercial-aircraft/global-market-forecast

[2] B. J. V. Da Silva, R. Morabito, D. S. Yamashita, and H. H. Yanasse, "Production scheduling of assembly fixtures in the aeronautical industry," *Comput. Ind. Eng.*, vol. 67, pp. 195–203, Jan. 2014.

[3] Z. Tian, X. Jiang, G. Tian, Z. Li, and W. Liu, "Knowledge-based lot-splitting optimization method for flexible job shops considering energy consumption," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 4864–4875, Jul. 2024.

[4] M. R. Garey, D. S. Johnson, and R. Sethi, "The complexity of flowshop and jobshop scheduling," *Math. Oper. Res.*, vol. 1, no. 2, pp. 117–129, May 1976.

[5] J.-q. Li et al., "Efficient multi-objective algorithm for the lot-streaming hybrid flowshop with variable sub-lots," *Swarm Evol. Comput.*, vol. 52, Feb. 2020, Art. no. 100600.

[6] R. Li, L. Wang, W. Gong, and F. Ming, "An evolutionary multitasking memetic algorithm for multi-objective distributed heterogeneous welding flow shop scheduling," *IEEE Trans. Evol. Comput.*, early access, Apr. 25, 2024, doi: 10.1109/TEVC.2024.3393620.

[7] H. Tang, J. Huang, C. Ren, Y. Shao, and J. Lu, "Integrated scheduling of multi-objective lot-streaming hybrid flowshop with AGV based on deep reinforcement learning," *Int. J. Prod. Res.*, vol. 63, no. 4, pp. 1275–1303, 2024.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.

[8] B. Lu, K. Gao, Y. Ren, D. Li, and A. Slowik, "Combining metaheuristics and Q-learning for scheduling lot-streaming hybrid flow shops with consistent sublots," *Swarm Evol. Comput.*, vol. 91, Dec. 2024, Art. no. 101731.

[9] W. Shao, Z. Shao, and D. Pi, "Modelling and optimization of distributed heterogeneous hybrid flow shop lot-streaming scheduling problem," *Expert Syst. Appl.*, vol. 214, Mar. 2023, Art. no. 119151.

[10] F. M. Defersha and S. B. Movahed, "Linear programming assisted (not embedded) genetic algorithm for flexible jobshop scheduling with lot streaming," *Comput. Ind. Eng.*, vol. 117, pp. 319–335, Mar. 2018.

[11] A. Bożek and F. Werner, "Flexible job shop scheduling with lot streaming and sublot size optimisation," *Int. J. Prod. Res.*, vol. 56, no. 19, pp. 6391–6411, Oct. 2018.

[12] G. Tian, W. Wang, H. Zhang, X. Zhou, C. Zhang, and Z. Li, "Multi-objective optimization of energy-efficient remanufacturing system scheduling problem with lot-streaming production mode," *Expert Syst. Appl.*, vol. 237, Mar. 2024, Art. no. 121309.

[13] H. Wang, T. Peng, X. Li, J. He, W. Liu, and R. Tang, "An integrated simulation–optimization method for flexible assembly job shop scheduling with lot streaming and finite transport resources," *Comput. Ind. Eng.*, vol. 200, Feb. 2025, Art. no. 110790.

[14] L. Li, "Research on discrete intelligent workshop lot-streaming scheduling with variable sublots under engineer to order," *Comput. Ind. Eng.*, vol. 165, Mar. 2022, Art. no. 107928.

[15] Y. Zhu, Q. Tang, L. Zhang, M. He, and J. Kapenda, "Improved multi-objective artificial bee colony algorithm for parallel machine lot-streaming scheduling problem with limited and unequal sublots," *Comput. Ind. Eng.*, vol. 183, Sep. 2023, Art. no. 109428.

[16] J. He and J. Li, "Deep reinforcement learning based on graph neural network for flexible job shop scheduling problem with lot streaming," in *Proc. 20th Int. Conf. Adv. Intell. Comput. Technol. Appl.*, 2024, pp. 85–95.

[17] S. Chen, X. Wang, Y. Wang, and X. Gu, "A knowledge-driven many-objective algorithm for energy-efficient distributed heterogeneous hybrid flowshop scheduling with lot-streaming," *Swarm Evol. Comput.*, vol. 91, Dec. 2024, Art. no. 101771.

[18] F. Liu et al., "A systematic survey on large language models for algorithm design," 2024, *arXiv:2410.14716*.

[19] C. Yang et al., "Large language models as optimizers," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 1–42.

[20] R. Lange, Y. Tian, and Y. Tang, "Large language models as evolution strategies," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2024, pp. 579–582.

[21] L. L. Custode, F. Caraffini, A. Yaman, and G. Iacca, "An investigation on the use of large language models for hyperparameter tuning in evolutionary algorithms," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2024, pp. 1838–1845.

[22] Y. Zhou et al., "Large language models are human-level prompt engineers," 2023, *arXiv:2211.01910*.

[23] H. Hao, X. Zhang, and A. Zhou, "Large language models as surrogate models in evolutionary algorithms: A preliminary study," 2024, *arXiv:2406.10675*.

[24] G. Jawahar, M. Abdul-Mageed, L. V. S. Lakshmanan, and D. Ding, "LLM performance predictors are good initializers for architecture search," 2024, *arXiv:2310.16712*.

[25] S. Bhambri, A. Bhattacharjee, D. Kalwar, L. Guan, H. Liu, and S. Kambhampati, "Extracting heuristics from large language models for reward shaping in reinforcement learning," 2024, *arXiv:2405.15194*.

[26] X. Wu, Y. Zhong, J. Wu, B. Jiang, and K. C. Tan, "Large language model-enhanced algorithm selection: Towards comprehensive algorithm representation," in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 5235–5244. [Online]. Available: https://doi.org/10.24963/ijcai.2024/579

[27] X. Jiang, Y. Wu, Y. Wang, and Y. Zhang, "Bridging large language models and optimization: A unified framework for text-attributed combinatorial optimization," 2024, *arXiv:2408.12214*.

[28] F. Liu et al., "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 1–23.

[29] Y. J. Ma et al., "Eureka: Human-level reward design via coding large language models," in *Proc. 12th Int. Conf. Learn. Represent.*, 2024, pp. 1–45.

[30] B. Romera-Paredes et al., "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, no. 7995, pp. 468–475, 2024.

[31] N. van Stein and T. Bäck, "LLaMEA: A large language model evolutionary algorithm for automatically generating metaheuristics," 2025, *arXiv:2405.20132*.

[32] R. Li, W. Gong, C. Lu, and L. Wang, "A learning-based memetic algorithm for energy-efficient flexible job-shop scheduling with type-2 fuzzy processing time," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 610–620, Jun. 2023.

[33] Z. Pan, L. Wang, J. Zheng, J. F. Chen, and X. Wang, "A learning-based multipopulation evolutionary optimization for flexible job shop scheduling problem with finite transportation resources," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1590–1603, Dec. 2023.

[34] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength pareto evolutionary algorithm," Comput. Eng. Netw. Lab., ETH Zurich, Zürich, Switzerland, Rep. 103, 2001.

[35] E. Taillard, "Benchmarks for basic scheduling problems," *Eur. J. Oper. Res.*, vol. 64, no. 2, pp. 278–285, 1993.

[36] Y. Li, C. Liao, L. Wang, Y. Xiao, Y. Cao, and S. Guo, "A reinforcement learning-artificial bee colony algorithm for flexible job-shop scheduling problem with lot streaming," *Appl. Soft Comput.*, vol. 146, Oct. 2023, Art. no. 110658.



Rui Li received the B.Eng. degree in information security and the M.Eng. degree in computer science and technology from China University of Geosciences, Wuhan, China, in 2020 and 2023, respectively. He is currently pursuing the Ph.D. degree in control science and engineering with the Department of Automation, Tsinghua University, Beijing, China.

His current research interests include distributed and complex shop scheduling with intelligent optimization and reinforcement learning.



Ling Wang (Senior Member, IEEE) received the B.Sc. degree in automation and the Ph.D. degree in control theory and control engineering from Tsinghua University, Beijing, China, in 1995 and 1999, respectively.

Since 1999, he has been with the Department of Automation, Tsinghua University, where he became a Full Professor in 2008. He has authored five academic books and more than 300 refereed papers. His current research interests include intelligent optimization and production scheduling.

Prof. Wang is a recipient of the National Natural Science Fund for Distinguished Young Scholars of China, the National Natural Science Award (Second Place) in 2014, the Science and Technology Award of Beijing City in 2008, and the Natural Science Award (First Place in 2003, and Second Place in 2007) nominated by the Ministry of Education of China. He is currently the Editor-in-Chief of *International Journal of Automation and Control*, *Swarm and Evolutionary Computation*, *Expert Systems with Applications*, and the Associate Editor of IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION.



Hongyan Sang (Member, IEEE) received the M.S. degree from the School of Computer Science, Liaocheng University, Liaocheng, China, in 2010, and the Ph. D. degree in industrial engineering from Huazhong University of Science Technology, Wuhan, China, in 2013.

Since 2003, she has been with the School of Computer Science, Liaocheng University, where she became a Professor in 2021. She has authored more than 100 refereed papers. Her current research interests include intelligent optimization and scheduling.

Prof. Sang is an Associate Editor of *Swarm and Evolutionary Computation*, and *Expert Systems With Applications*. She is also an editorial board of *Engineering Applications of Artificial Intelligence*.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.



Lizhong Yao received the B.S. degree in industrial automation from Chongqing University of Science and Technology, Chongqing, China, in 2009, the M.S. degree in detection technology and automatic equipment from Xian Shiyou University, Xian, China, in 2013, and the Ph.D. degree in mechanical and electronic engineering from Sichuan University, Chengdu, China, in 2020.

He is currently an Associate Professor with the College of Physics and Electronic Engineering, Chongqing Normal University, Chongqing. His current research interests include industrial process modeling and multitasking optimization.



Lijun Pan received the Ph.D. degree in logistics engineering from Central South University, Changsha, China, in 2012.

He is currently a Professor with the Hunan Institute of Engineering, Xiangtan, China. His main research interests include transport system optimization and intelligent computing.

Authorized licensed use limited to: BEIJING UNIVERSITY OF TECHNOLOGY. Downloaded on October 13,2026 at 01:20:47 UTC from IEEE Xplore. Restrictions apply.